

Instituto Federal do Paraná
Campus Pinhais

Bacharelado em Ciência da Computação

Relatório do Trabalho

*Método do Gradiente Conjugado — Implementação e Análise de
Desempenho*

Pinhais
2026

1. Introdução

Este relatório descreve o desenvolvimento de um programa em C++ para resolver sistemas de equações lineares da forma $Ax = b$, utilizando o Método do Gradiente Conjugado (CG). O trabalho foi desenvolvido individualmente e segue os requisitos descritos na especificação do trabalho.

O objetivo principal é implementar o método, medir o desempenho da versão não otimizada, aplicar otimizações de código, e comparar os resultados entre as duas versões. Os testes foram realizados para matrizes esparsas de 6 tamanhos diferentes, com 7 e 27 bandas cada.

2. Descrição da Implementação

2.1 Organização dos Arquivos

O projeto foi organizado em apenas dois arquivos principais de código, conforme a proposta de manter a implementação simples e direta:

Arquivo	Descrição
main.cpp	Versão não otimizada: geração da matriz, algoritmo CG e medição de tempo em um único arquivo
mainoptimized.cpp	Versão otimizada: mesmo algoritmo com AVX2 e loop unrolling aplicados ao matvec e dot product
Makefile	Compila ambas as versões: cgSolver (flag -O0) e cgSolverOpt (flags -O2 -mavx2 -mfma)

2.2 Representação da Matriz Esparsa

A matriz do sistema é grande e esparsa — a maioria dos elementos é zero. Guardá-la completa seria inviável: para $N = 1.048.576$, isso exigiria mais de 8 TB de RAM. Por isso, armazenamos apenas as diagonais não-nulas usando a seguinte estrutura:

struct SparseMatrix — campos: n (dimensão), num_diags (7 ou 27), offsets (distância de cada diagonal à principal), diags (vetores com os valores de cada diagonal).

Para 7 bandas, os offsets são $\{-3, -2, -1, 0, +1, +2, +3\}$. Para 27 bandas, variam de -13 a +13. O uso de memória cai de $O(N^2)$ para $O(N \times \text{num_diags})$ — no pior caso ($N=1.048.576$, 27 bandas), isso representa apenas 256 MB.



INSTITUTO FEDERAL
Paraná
Campus Pinhais

2.3 Geração da Matriz

A matriz é gerada de forma aleatória, mas respeitando duas propriedades exigidas pelo método:

- Simetria: valores das diagonais acima da principal são gerados aleatoriamente e espelhados nas diagonais abaixo.
- Positividade definida: garantida por dominância diagonal — a diagonal principal de cada linha recebe a soma dos valores absolutos de todos os outros elementos da mesma linha, acrescida de 1,0.
- Reprodutibilidade: seed fixa (42) garante que sempre geramos a mesma matriz, permitindo comparações justas entre as versões.

2.4 Algoritmo do Gradiente Conjugado

O algoritmo resolve $Ax = b$ de forma iterativa, partindo de $x = 0$ e melhorando a solução a cada iteração. Os passos de cada iteração são:

1. Calcula $Ap = A \times p$ (produto matriz-vetor — operação mais custosa)
2. Calcula $\alpha = r^T r / p^T Ap$ (tamanho do passo)
3. Atualiza solução: $x = x + \alpha \times p$
4. Atualiza resíduo: $r = r - \alpha \times Ap$
5. Verifica convergência: se $\|r\| < 1e-10$, encerra
6. Calcula $\beta = r_{\text{novo}}^T r_{\text{novo}} / r^T r$ (correção de direção)
7. Atualiza direção: $p = r + \beta \times p$

2.5 Medição de Tempo e Memória

A instrumentação foi feita com `std::chrono::high_resolution_clock`, medindo em milissegundos:

- Tempo de geração da matriz e vetor b
- Tempo total do loop do Gradiente Conjugado
- Tempo acumulado do produto matriz-vetor (matvec)
- Tempo acumulado dos produtos internos (dot products)

Além do tempo, o programa calcula analiticamente: RAM alocada total, tráfego de memória por operação, tráfego total durante o CG e largura de banda efetiva (GB/s).



INSTITUTO FEDERAL
Paraná
Campus Pinhais

3. Otimizações Implementadas

3.1 Por que otimizar o Matvec?

Os dados da versão não otimizada mostram que o produto matriz-vetor (matvec) consome entre 60% e 87% do tempo total do CG, dependendo do número de bandas. É o gargalo principal do programa e, portanto, o foco das otimizações.

3.2 AVX2 — Processamento Vetorial

AVX2 (Advanced Vector Extensions 2) é um conjunto de instruções do processador que permite operar em 4 valores double simultaneamente usando registradores de 256 bits. Em vez de fazer uma multiplicação por vez, fazemos 4 de uma só vez.

A instrução `_mm256_fmadd_pd` realiza multiplicação e adição em uma única operação (Fused Multiply-Add), reduzindo o número total de instruções executadas.

3.3 Loop Unrolling — Desenrolamento de Laços

No loop interno do matvec, em vez de processar 1 elemento por iteração, processamos 4 com AVX e tratamos os elementos restantes em blocos de 2. Isso reduz o overhead de verificação de condição e incremento do loop, e facilita a execução fora de ordem pelo processador.

3.4 Flag de Compilação -O2

A versão otimizada é compilada com `-O2 -mavx2 -mfma`, enquanto a não otimizada usa `-O0`. A flag `-O2` ativa otimizações automáticas do compilador como inlining de funções, eliminação de código morto e reorganização de instruções.

4. Resultados de Desempenho

4.1 Versão Não Otimizada — 7 Bandas

N	Iter	Ger (ms)	CG Total (ms)	Matvec (ms)	Matvec %	Dot (ms)	BW (GB/s)
1024	28	0.288	1.684	1.006	59.7%	0.375	3.04
4096	30	0.861	6.008	3.841	63.9%	1.261	3.66
16384	32	2.902	15.586	9.795	62.8%	3.186	6.02
65536	33	11.872	57.774	36.884	63.8%	12.534	6.69
262144	35	25.359	241.570	155.152	64.2%	52.250	6.79
1048576	37	100.952	1073.294	683.028	63.6%	234.270	6.46

4.2 Versão Não Otimizada — 27 Bandas

N	Iter	Ger (ms)	CG Total (ms)	Matvec (ms)	Matvec %	Dot (ms)	BW (GB/s)
1024	23	1.042	3.747	3.172	84.6%	0.292	2.06
4096	27	2.960	11.283	9.732	86.2%	0.885	3.21
16384	26	8.503	30.557	26.308	86.1%	2.513	4.57
65536	27	23.851	125.103	107.359	85.8%	10.523	4.64
262144	27	94.223	508.619	439.859	86.5%	40.548	4.56
1048576	28	383.949	2223.273	1924.467	86.6%	175.003	4.33

4.3 Versão Otimizada — 7 Bandas

N	Iter	Ger (ms)	CG Total (ms)	Matvec (ms)	Matvec %	Dot (ms)	BW (GB/s)
1024	28	0.202	0.274	0.092	33.7%	0.057	18.73
4096	30	0.712	1.223	0.495	40.5%	0.220	17.96
16384	32	2.607	3.975	1.915	48.2%	0.644	23.59
65536	33	7.907	10.937	6.309	57.7%	1.539	35.36
262144	35	25.846	68.881	43.538	63.2%	8.039	23.82
1048576	37	87.718	395.379	245.790	62.2%	49.567	17.55

4.4 Versão Otimizada — 27 Bandas

N	Iter	Ger (ms)	CG Total (ms)	Matvec (ms)	Matvec %	Dot (ms)	BW (GB/s)
1024	23	0.592	0.433	0.284	65.5%	0.041	17.82
4096	27	2.368	2.309	1.761	76.3%	0.176	15.70
16384	26	8.042	4.366	3.416	78.2%	0.296	31.98
65536	27	19.945	32.079	27.365	85.3%	1.410	18.08
262144	27	79.284	150.079	127.447	84.9%	6.272	15.46
1048576	28	313.948	961.942	834.417	86.7%	40.461	10.01

5. Comparação: Não Otimizado vs Otimizado

5.1 Tabela Comparativa — 7 Bandas

N	Bandas	CG N.Otim (ms)	CG Otim (ms)	Ganho CG	BW N.Otim (GB/s)	BW Otim (GB/s)
1024	7	1.684	0.274	6.1x	3.04	18.73
4096	7	6.008	1.223	4.9x	3.66	17.96
16384	7	15.586	3.975	3.9x	6.02	23.59
65536	7	57.774	10.937	5.3x	6.69	35.36
262144	7	241.570	68.881	3.5x	6.79	23.82
1048576	7	1073.294	395.379	2.7x	6.46	17.55

5.2 Tabela Comparativa — 27 Bandas

N	Bandas	CG N.Otim (ms)	CG Otim (ms)	Ganho CG	BW N.Otim (GB/s)	BW Otim (GB/s)
1024	27	3.747	0.433	8.7x	2.06	17.82
4096	27	11.283	2.309	4.9x	3.21	15.70
16384	27	30.557	4.366	7.0x	4.57	31.98
65536	27	125.103	32.079	3.9x	4.64	18.08
262144	27	508.619	150.079	3.4x	4.56	15.46
1048576	27	2223.273	961.942	2.3x	4.33	10.01

5.3 Resumo dos Ganhos

Caso	Tempo CG: Não Otim → Otim	Largura de Banda	Ganho
N=1.048.576 7 bandas	1073 ms → 395 ms	6.46 → 17.55 GB/s	2.7x
N=1.048.576 27 bandas	2223 ms → 962 ms	4.33 → 10.01 GB/s	2.3x
N=65536 7 bandas	57.8 ms → 10.9 ms	6.69 → 35.36 GB/s	5.3x
N=16384 27 bandas	30.6 ms → 4.4 ms	4.57 → 31.98 GB/s	7.0x

6. Análise dos Resultados

6.1 Convergência

O método convergiu com sucesso em todos os 24 casos de teste (6 tamanhos $\times$ 2 configurações de bandas $\times$ 2 versões). O número de iterações cresce muito lentamente com N — entre 23 e 37 iterações mesmo para $N = 1.048.576$. Isso confirma que o Gradiente Conjugado é eficiente para matrizes bem condicionadas como as geradas neste trabalho.

A versão otimizada produziu exatamente os mesmos resíduos finais que a não otimizada, confirmando que as otimizações não alteraram a correção matemática do algoritmo.

6.2 Distribuição do Tempo

Em todas as configurações testadas, o matvec domina o tempo de execução:

- 7 bandas (não otimizado): matvec consome entre 59% e 64% do tempo do CG.
- 27 bandas (não otimizado): matvec consome entre 84% e 87% do tempo do CG.
- Com otimização: a participação do matvec cai para 33–62% em 7 bandas, pois o AVX2 o acelera mais do que o dot product.

O aumento da participação do matvec com 27 bandas faz sentido: ele realiza quase 4 vezes mais operações por linha do que com 7 bandas, enquanto os dot products sempre custam $O(N)$ independente das bandas.

6.3 Ganho das Otimizações

O ganho de desempenho varia conforme o tamanho da matriz e o número de bandas:

- Para matrizes pequenas ($N=1024$), o ganho é maior (6x) porque os dados cabem inteiramente no cache L1/L2, e o AVX aproveita ao máximo.
- Para matrizes grandes ($N=1.048.576$), o ganho cai para 2,3x–2,7x porque o gargalo passa a ser a largura de banda da RAM — os dados não cabem mais no cache.
- O pico de 7x de ganho ocorre em casos intermediários ($N=16384$ com 27 bandas), onde o AVX é aproveitado sem sofrer tanto com cache misses.

6.4 Largura de Banda

A largura de banda efetiva aumentou significativamente com as otimizações:

- Não otimizado: entre 2 e 7 GB/s — muito abaixo da capacidade teórica da RAM.
- Otimizado: entre 10 e 35 GB/s — o AVX2 permite usar a RAM de forma muito mais eficiente.

Os valores mais altos (>20 GB/s) ocorrem quando os dados cabem no cache (matrizes menores), o que explica a variação. Para $N=1.048.576$, mesmo com otimização, a largura cai porque o volume de dados excede a capacidade do cache e o programa fica limitado pela velocidade da RAM.



INSTITUTO FEDERAL
Paraná
Campus Pinhais

7. Desafios Enfrentados

Durante o desenvolvimento do trabalho, alguns desafios técnicos e conceituais foram enfrentados durante o desenvolvimento do trabalho.

O primeiro deles foi a complexidade matemática do Método do Gradiente Conjugado. Apesar do algoritmo seguir uma sequência bem definida de passos, entender o papel de cada variável, especialmente os coeficientes alpha e beta e a relação entre o resíduo e a direção de busca, exigiu tempo considerável de estudo e experimentação. Fiquei travado em muitas parte do código até que fosse entendido qual o próximo passo teria que seguir.

O segundo desafio foi o uso de bibliotecas e recursos que não havia trabalhado antes. A biblioteca <chrono> para medição de tempo de alta precisão e, principalmente, as instruções vetoriais AVX2 com <immintrin.h> foram utilizadas pela primeira vez neste trabalho. O uso de intrinsics AVX2, como `_mm256_loadu_pd`, `_mm256_fmadd_pd` e a redução horizontal de registradores, demandou pesquisa detalhada sobre o funcionamento interno dessas instruções, sobre alinhamento de memória e sobre como o processador opera com registradores de 256 bits. Isso tornou o processo de desenvolvimento mais lento e exigiu maior cautela para garantir a correção dos resultados.

O terceiro contratempo foi a impossibilidade de utilizar a ferramenta Likwid, sugerida na especificação do trabalho para monitoramento de hardware. Todo o desenvolvimento foi realizado em ambiente WSL (Windows Subsystem for Linux), que por limitações impostas pelo Windows não permite acesso a camadas de privilégio alta, operando a nível de Kernel. O Likwid depende exatamente desse nível de acesso para exibir métricas de hardware como contadores de cache e largura de banda real do processador. Como alternativa, o monitoramento de memória foi realizado de forma analítica diretamente no código, calculando o volume de dados movimentados e a largura de banda efetiva com base nos tempos medidos e nos tamanhos das estruturas alocadas.



INSTITUTO FEDERAL

Paraná

Campus Pinhais

8. Conclusão

O trabalho foi concluído com sucesso. A implementação do Método do Gradiente Conjugado está funcionando perfeitamente para todos os casos de teste, convergindo em poucas dezenas de iterações independentemente do tamanho da matriz.

As otimizações com AVX2, loop unrolling e a flag -O2 proporcionaram ganhos reais e na otimização: o tempo do CG foi reduzido em 2,3x a 7x dependendo do caso, e a largura de banda efetiva aumentou em média 3,5x. A versão otimizada resolve o maior sistema ($N=1.048.576$, 27 bandas) em menos de 1 segundo, contra mais de 2 segundos na versão não otimizada.

O principal aprendizado do trabalho é que, para problemas numéricos com matrizes esparsas, o gargalo de desempenho está quase sempre no produto matriz-vetor, e técnicas de vetorização (AVX), nesse caso, foram as mais eficazes para acelerá-lo.